2025 计算机体系结构-作业 1

学号：2023212351 姓名：邓杰湧

1. Amdahl 定律

(1) 你正在开发一项新的增强技术，该技术能使特定类型的指令获得 2.5 倍的加速比。若希望程序整体获得 10%的加速比，那么以程序原始执行时间为衡量标准，这类可加速的指令的处理时间占整个程序的执行时间的百分比是多少？

使用阿姆达尔定律：

$$S_{整体} = \frac{1}{(1-\alpha) + \frac{\alpha}{S_p}}$$

S 整体=1.1, $Sp = 2.5$, 带入可得$\alpha = 0.1515$

这类可加速的指令的处理时间需占整个程序执行时间的 15.15%

(2) 对于(1)中确定的程序，若上述增强技术能提供无限大的加速比，则程序能获得的最大可能加速比是多少？

即$S_p \to \infty$, $S_{极限} = \frac{1}{1-\alpha}$, 将其代入简化公式：

$$S_{极限} = \frac{1}{0.8485} \approx 1.178$$

当增强技术提供无限大加速比时，程序能获得的最大可能加速比约为 1.18 倍

(3) 现有两项可用的增强技术：第一项可对程序 40% 的执行时间进行优化，加速比为 1.5；第二项可对程序 25%的执行时间进行优化，但其加速比尚未确定。由于条件限制，只能实现其中一项技术。若要使第二项技术的优化效果优于第一项，其所需的加速比至少为多少？

第一项技术：

已知第一项技术的参数：

被优化部分占比$\alpha_1 = 40\% = 0.4$；

该部分加速比$S_{p1} = 1.5$。

代入公式计算整体加速比S_1=1.1538

已知第二项技术的参数：

被优化部分占比 $\alpha_2 = 25\% = 0.25$；

设该部分加速比为 S_{p2}；

要求：第二项的整体加速比 $S_2 > S_1 \approx 1.1538$

建立不等式：$\frac{1}{0.75+\frac{0.25}{S_{p2}}} > 1.1538$

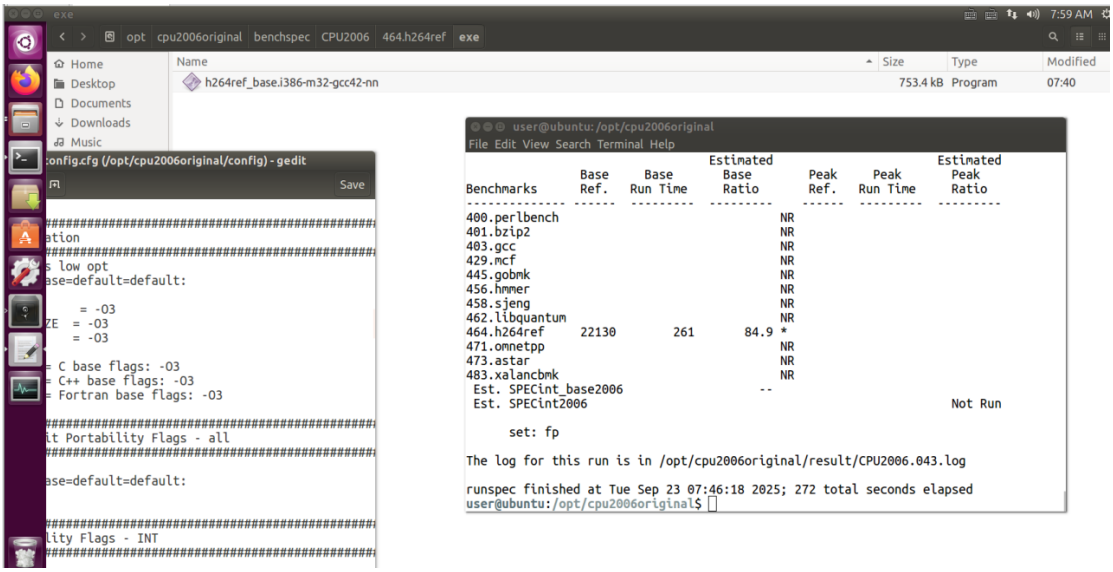
为使第二项技术的优化效果优于第一项，其所需的加速比至少约为 2.14

2. **基准测试：**

从所提供的虚拟机[1]中运行 SPEC2006 基准测试集中的任意 3 个程序，并分别配置编译优化选项为 -O1，-O2，-O3。请将生成的各个程序的大小，以及使用 **ref 作为输入**的运行时间、可执行文件的大小填入下面的表格中。

Workload name	程序大小	运行时间
464.h264ref -O1	565.2kb	255s
464.h264ref -O2	573.3kb	270s
464.h264ref -O3	753.4kb	261s
429.mcf -O1	17.3kb	154s
429.mcf -O1	21.4kb	143s
429.mcf -O1	21.4kb	133s
401.bzip2 -O1	67.6KB	313s
401.bzip2 -O2	67.6KB	310s
401.bzip2 -O3	83.9KB	312s

[1] ubuntu_spec : https://pan.baidu.com/s/1R1aqsOP9vTWKce5k-aBZ-Q?pwd=HFUT 提取码: HFUT



3. 缓存性能：

组相联 Cache 的失效率比相同容量的直接相联映像 Cache 的失效率低。由此能否得出结论：采用组相联一定能带来性能上的提高？为什么？

不能得出 “采用组相联一定能带来性能上的提高” 的结论。虽然组相联 Cache 的失效率低于相同容量的直接相联 Cache，但**Cache 的整体性能由 “平均访问时间” 决定**，而平均访问时间不仅与失效率相关，还取决于 “命中时间” “失效开销” 等关键因素。组相联 Cache 在降低失效率的同时，可能导致其他性能指标恶化，反而可能抵消甚至超过失效率降低带来的收益。组相联 Cache 的性能短板：命中时间显著增加

平均访问时间（AAT）=命中时间+失效率×失效开销

例如：

直接相联：命中时间 1ns，失效率 5%，失效开销 10ns → 平均访问时间 = 1 + 5%×10=1.5ns；

4 路组相联：命中时间 1.5ns，失效率 3%，失效开销 10ns → 平均访问时间 = 1.5 + 3%×10=1.8ns。

此时组相联虽失效率更低，但因失效开销本身较小，失效率降低的收益不足以抵消命中时间增加的损失，性能反而下降。

4. 替换算法：

考虑一个循环访问 4 个地址：A、B、C、D 的访问序列（ABCDABCDABCD…）。基于这个访问序列，请回答不同的缓存替换算法在一个 2 个条目的全相联缓存中的几个问题：

（1） 根据 LRU 替换算法，完成下面的表格，并计算长期缺失率。(注意每次访问是命中(H)还是缺失(M)）

地址	A	B	C	D	A	B	C	D	A	B	C	D	…
块-0	A	A	C	C	A	A	C	C	A	A	C	C	…
块-1	空	B	B	D	D	B	B	D	D	B	B	D	…
H/M	M	M	M	M	M	M	M	M	M	M	M	M	…

长期缺失率为 100%

（2） 基于理想的替换算法，完成下面的表格，并计算长期缺失率。

地址	A	B	C	D	A	B	C	D	A	B	C	D	…
块-0	A	A	A	A	A	B	C	C	C	C	C	D	…
块-1	空	B	C	D	D	D	D	D	A	B	B	B	…
H/M	M	M	M	M	H	M	M	H	M	M	H	M	…

理想的替换算法每次替换时保留序列中下一次访问最近的地址，长期缺失率为 66.6%